

Sistemas Distribuídos – Relatório

Universidade Federal do Ceará – Campus Quixadá

Prática: Laboratório – FastAPI

Aluno(a): Mônica Maria Rodrigues

Maryana Moraes

Professor: Rafael Braga

1. Objetivo

O objetivo desta prática foi desenvolver uma **API RESTful utilizando o framework FastAPI**, na linguagem Python, com persistência de dados em um arquivo XML. A aplicação permite o gerenciamento de um estoque de aparelhos eletrônicos, oferecendo operações de criação, leitura, atualização, remoção e transferência de aparelhos entre depósitos por meio de requisições HTTP.

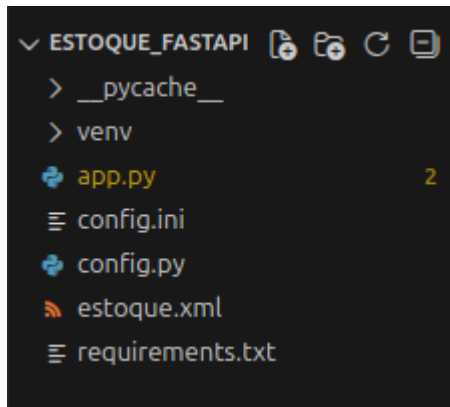
A atividade tem como finalidade reforçar os conceitos de **sistemas distribuídos, arquitetura REST, comunicação cliente-servidor e serviços web**.

2. Ferramentas Utilizadas

- Python 3
- FastAPI
- Uvicorn
- Pydantic
- XML (Extensible Markup Language)
- ConfigParser
- Swagger UI
- Terminal Linux (Ubuntu)
- Ambiente Virtual Python (venv)

3. Estrutura do Projeto

O projeto foi organizado conforme a estrutura abaixo:



- **app.py**: arquivo principal contendo a implementação da API;
- **config.py**: responsável pela criação do arquivo de configuração;
- **config.ini**: define o nome do arquivo XML utilizado;
- **estoque.xml**: arquivo responsável pelo armazenamento dos dados;
- **requirements.txt**: lista de dependências do projeto;
- **venv/**: ambiente virtual Python.

4. Configuração do Ambiente e Dependências

Inicialmente, foi criado um ambiente virtual para isolamento das dependências do projeto, devido às restrições do sistema operacional quanto à instalação global de pacotes Python.

Comandos utilizados:

```
-> python3 -m venv venv  
-> source venv/bin/activate  
-> pip install -r requirements.txt
```

5. Implementação da API

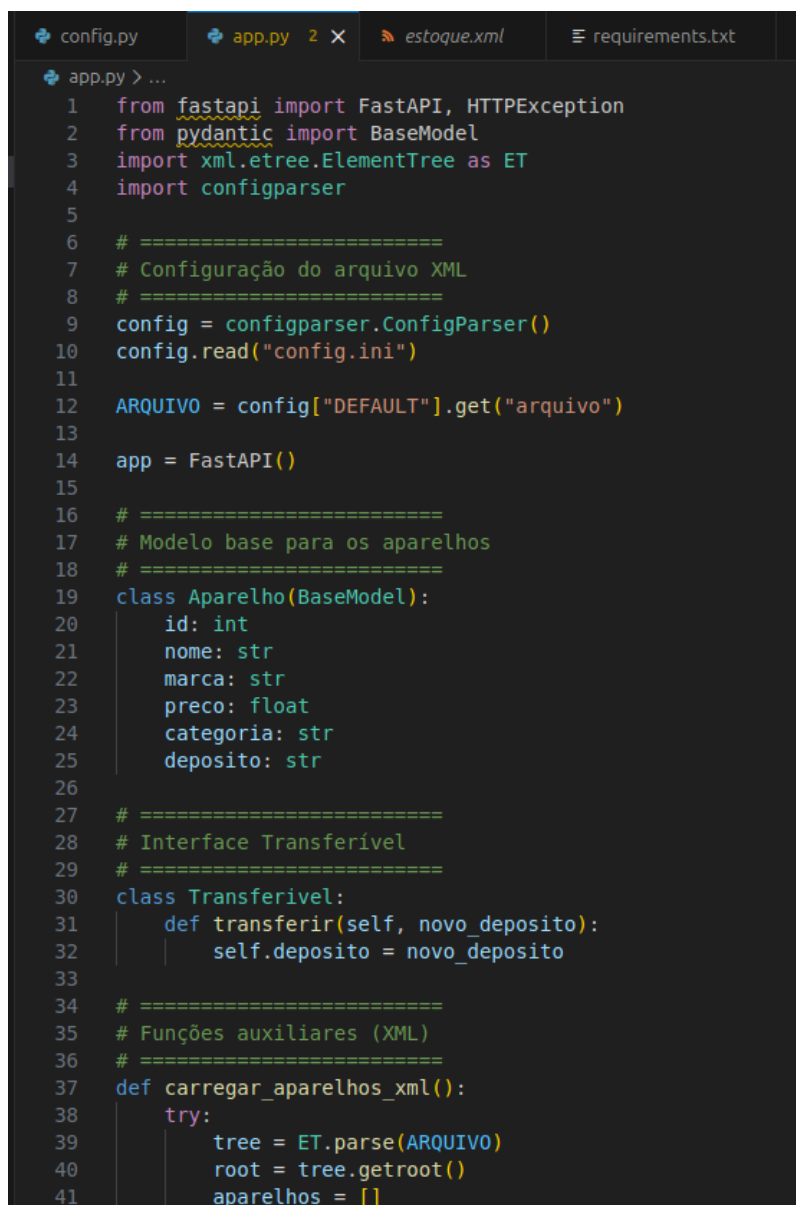
A API foi implementada utilizando o FastAPI, framework que facilita a criação de serviços REST e gera automaticamente uma documentação interativa.

Foi definido um modelo de dados chamado **Aparelho**, contendo os seguintes atributos:

- id
- nome
- marca
- preço
- categoria
- depósito

A persistência dos dados é realizada por meio de um arquivo XML, que é criado automaticamente caso não exista.

-> arquivo **app.py** (uma parte do arquivo)



```
1  from fastapi import FastAPI, HTTPException
2  from pydantic import BaseModel
3  import xml.etree.ElementTree as ET
4  import configparser
5
6  # =====
7  # Configuração do arquivo XML
8  # =====
9  config = configparser.ConfigParser()
10 config.read("config.ini")
11
12 ARQUIVO = config["DEFAULT"].get("arquivo")
13
14 app = FastAPI()
15
16 # =====
17 # Modelo base para os aparelhos
18 # =====
19 class Aparelho(BaseModel):
20     id: int
21     nome: str
22     marca: str
23     preco: float
24     categoria: str
25     deposito: str
26
27 # =====
28 # Interface Transferível
29 # =====
30 class Transferivel:
31     def transferir(self, novo_deposito):
32         self.deposito = novo_deposito
33
34 # =====
35 # Funções auxiliares (XML)
36 # =====
37 def carregar_aparelhos_xml():
38     try:
39         tree = ET.parse(ARQUIVO)
40         root = tree.getroot()
41         aparelhos = []
```

5.1 Manipulação do Arquivo XML

Foram implementadas funções auxiliares para carregar e salvar os dados no arquivo XML. Essas funções garantem que todas as operações realizadas pela API sejam refletidas corretamente no arquivo de persistência.

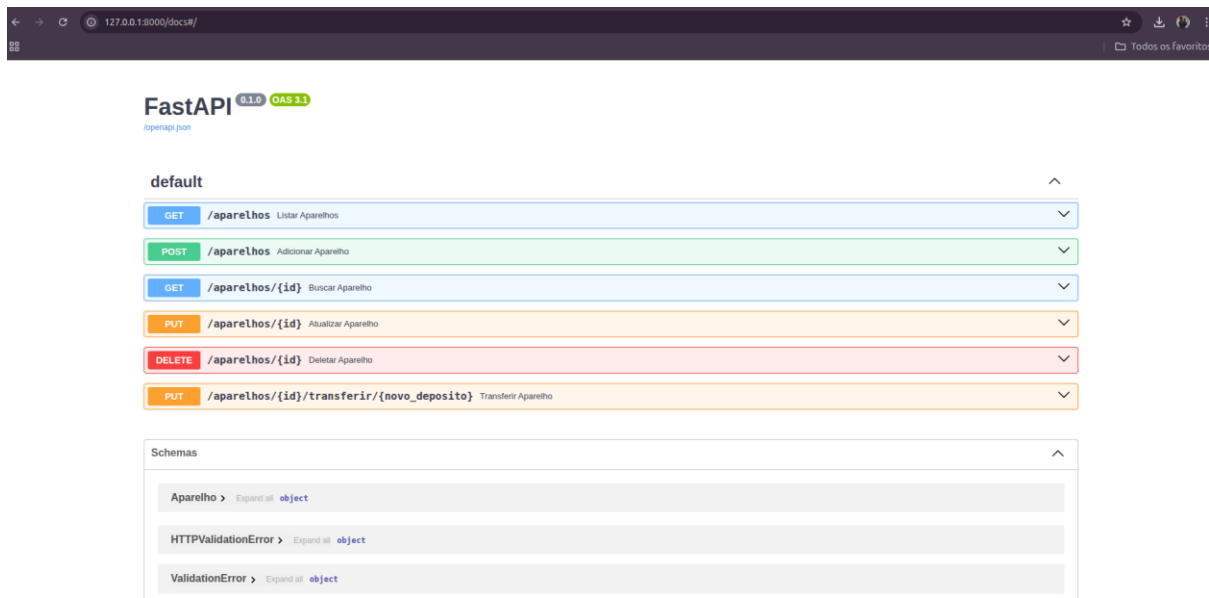
-> arquivo estoque.xml

```
maryana-moraes@maryana-moraes-Aspire-A515-57:~/Documentos/UFC/8_Semestre/Sistemas Distribuidos/atividades/Laboratórios/lab 3/estoque_fastapi$ cat estoque.xml
<?xml version='1.0' encoding='utf-8'?>
<estoque><aparelho><id>0</id><nome>string</nome><marca>string</marca><preco>0.0</preco><categoria>string</categoria><deposito>string</deposito></aparelho><aparelho><id>1</id><no
me>Lab FasAPI</nome><marca>UFC</marca><preco>50.0</preco><categoria>Laboratórios</categoria><deposito>Quixadá</deposito></aparelho></estoque>maryana-moraes@maryana-moraes-Aspire
```

5.2 Endpoints da API

A aplicação disponibiliza os seguintes endpoints REST:

- **POST /aparelhos** – adiciona um novo aparelho ao estoque;
- **GET /aparelhos** – lista todos os aparelhos cadastrados;
- **GET /aparelhos/{id}** – retorna um aparelho específico;
- **PUT /aparelhos/{id}** – atualiza os dados de um aparelho;
- **DELETE /aparelhos/{id}** – remove um aparelho do estoque;
- **PUT /aparelhos/{id}/transferir/{novo_deposito}** – transfere um aparelho para outro depósito.



6. Execução da Aplicação

Após a implementação, execute o seguinte comando:

-> python config.py

Executando o servidor:

-> uvicorn app:app --reload

O serviço ficou disponível no endereço:

-> <http://127.0.0.1:8000/docs>

```
(venv) maryana-moraes@maryana-moraes-Aspire-A515-57:~/Documentos/UFC/8_Semestre/Sistemas Distribuidos/atividades/Laboratórios/Lab 3/estoque_fastapi$ uvicorn app:app --reload
INFO: Will watch for changes in these directories: ['/home/maryana-moraes/Documentos/UFC/8_Semestre/Sistemas Distribuidos/atividades/Laboratórios/Lab 3/estoque_fastapi']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [14122] using StatReload
INFO: Started server process [14139] using StatReload
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:56936 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:56936 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:49344 - "POST /aparelhos HTTP/1.1" 201 Created
INFO: 127.0.0.1:55880 - "POST /aparelhos HTTP/1.1" 201 Created
INFO: 127.0.0.1:34198 - "GET /aparelhos/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:46684 - "GET /aparelhos/1 HTTP/1.1" 200 OK
```

7. Testes da API

Os testes foram realizados por meio da interface **Swagger UI**, acessada pelo endereço:

<http://127.0.0.1:8000/docs>

Por meio dessa interface, foi possível testar todos os endpoints da aplicação, inserindo, consultando, atualizando e removendo dados do estoque.

The screenshot shows the Swagger UI interface. At the top, there's a tab labeled "Request body" with a red "required" indicator. To the right, a dropdown menu shows "application/json". Below this, there are two tabs: "Edit Value" (selected) and "Schema". The "Edit Value" tab contains a JSON object:

```
{  "id": 1,  "nome": "Lab FasAPI",  "marca": "UFC",  "preco": 50,  "categoria": "Laboratórios",  "deposito": "Quixadá"}
```

 At the bottom, there are two buttons: "Execute" (highlighted in blue) and "Clear".

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/aparelhos' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 1,
    "nome": "Lab FasAPI",
    "marca": "UFC",
    "preco": 50,
    "categoria": "Laboratórios",
    "deposito": "Quixadá"
  }'
```

Request URL

http://127.0.0.1:8000/aparelhos

Server response

Code	Details
201	Response body <pre>{ "erro": false, "message": "Aparelho adicionado com sucesso" }</pre> Response headers <pre>content-length: 58 content-type: application/json date: Mon, 15 Dec 2025 18:09:47 GMT server: uvicorn</pre>

Após as operações, foi verificado que os dados estavam corretamente armazenados no arquivo XML.

```
AS15-57:~/Documentos/UFC/8_Semestre/Sistemas Distribuidos/atividades/Laboratórios/lab 3/estoque_fastapi$ cat estoque.xml
<?xml version='1.0' encoding='utf-8'?>
<estoque><aparelho><id>0</id><nome>string</nome><marca>string</marca><preco>0.0</preco><categoria>string</categoria><deposito>string</deposito></aparelho><aparelho><id>1</id><nome>Lab FasAPI</nome><marca>UFC</marca><preco>50.0</preco><categoria>Laboratórios</categoria><deposito>Quixadá</deposito></aparelho></estoque>maryana-moraes@maryana-moraes-Aspire
```

8. Conclusão

A prática foi concluída com sucesso, permitindo o desenvolvimento de uma API REST funcional utilizando FastAPI e Python. A aplicação demonstrou corretamente os conceitos de comunicação via HTTP, persistência de dados e organização de serviços distribuídos.

O uso do FastAPI facilitou a criação dos endpoints e forneceu uma documentação automática, tornando a aplicação simples de testar e manter.